VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



REQUIREMENTS SPECIFICATION

SUBJECT: INTELLIGENT SYSTEMS (CO5119)

TOPIC: A HYBRID PRODUCT RECOMMENDER FOR AN ELECTRONICS E-COMMERCE PLATFORM USING ASSOCIATION RULE MINING AND WORD EMBEDDINGS

Supervisor: **Assoc. Prof. Dr. Quản Thành Thơ**

Students:

Name	ID	Email
Phan Hữu Thành	2670371	thanh.phanbluewhales@hcmut.edu.vn
Trần Anh Khoa	2670309	khoa.trana@hcmut.edu.vn

Ho Chi Minh City, 10/2026

Contents

Scope of this document **1**

1 Role and Benefits of Stakeholders **2**
1.1 Product Buyers . . . 2
1.2 Administrators . . . 3
1.3 Data Scientists . . . 3

2 Functional Requirements **5**
2.1 Requirements for Product Buyers . . . 5
2.2 Requirements for Administrators . . . 5
2.3 Requirements for Data Scientists . . . 6
2.4 Requirements for the Data-Warehouse Pipeline . . . 7

3 General Use-Case Model **9**
3.1 Use-case catalogue . . . 11
3.2 Requirement-to-use-case traceability . . . 12
3.3 Main flows of the central use cases . . . 13

4 Dataset Description and Project Integration **17**
4.1 Dataset overview . . . 17
4.2 Files and fields . . . 17
4.2.1 Customers . . . 17
4.2.2 Orders . . . 18
4.2.3 Order items . . . 18
4.2.4 Order payments . . . 19
4.2.5 Products . . . 19
4.2.6 Sellers . . . 20
4.2.7 Order reviews . . . 21
4.2.8 Geolocation . . . 21
4.2.9 Category translations . . . 22
4.3 Project data model . . . 22
4.4 Adapting the data model to the project . . . 26

5 Interactive Prototype and Interface Mockups **28**
5.1 Buyer interface and explained recommendations . . . 28
5.1.1 Storefront and featured products . . . 28
5.1.2 Product catalogue . . . 29
5.1.3 Search results . . . 30
5.1.4 Product details . . . 31

5.1.5 Frequently bought together 32
5.1.6 Similar products 33
5.1.7 Shopping cart and basket recommendations 34
5.1.8 Checkout 35
5.1.9 Order confirmation 36
5.2 Administration dashboard and order inspection 38
5.2.1 Catalogue management 38
5.2.2 Product information form 38
5.2.3 Sales overview 39
5.2.4 Order browsing and selection 40
5.2.5 Order inspection 41
5.3 Data-scientist workspace 43
5.3.1 Dataset and model versions 43
5.4 Prototype boundary and remaining implementation 44

References **45**

List of Figures

1 General use-case diagram of the recommender-enabled storefront. 10
2 Adapted project data model and its entity relationships. 23
3 Main storefront with featured products and access to the collection. 29
4 Product catalogue with category and sorting controls. 30
5 Keyword search results for smartphones. 31
6 Selected smartphone with product information and an add-to-bag action. . 32
7 Nova X1 product page with explained complementary suggestions. 33
8 Nova X1 product page with four similar smartphone alternatives. 34
9 Shopping bag with selected products, subtotal and basket suggestions. . . . 35
10 Demo checkout with minimal contact information and order total. 36
11 Simulated order confirmation after completing checkout. 37
12 Administrator product list with catalogue-management actions. 38
13 Product-entry form populated with demonstration values. 39
14 Sales overview with period selection and sample indicators. 40
15 Searchable order records linked to inspection pages. 41
16 Sample order details with purchase-time quantities and prices. 42
17 Sample dataset and model versions in the data-scientist workspace. 43

List of Tables

1 Summary of stakeholders, their touch-points and primary benefits. 2
2 Functional requirements for product buyers. 5
3 Functional requirements for administrators. 6
4 Functional requirements for data scientists. 6
5 Functional requirements for the data-warehouse pipeline. 7
6 Use-case catalogue. 11
7 Functional requirement-to-use-case traceability. 12
8 Fields in the customers file. 17
9 Fields in the orders file. 18
10 Fields in the order items file. 19
11 Fields in the order payments file. 19
12 Fields in the products file. 20
13 Fields in the sellers file. 20
14 Fields in the order reviews file. 21
15 Fields in the geolocation file. 21
16 Fields in the category translations file. 22
17 Project model attributes: customer identity and location. 24

18 Project model attributes: product catalogue and categories. 24
19 Project model attributes: orders, purchased items and sellers. 25
20 Project model attributes: payments and reviews. 26
21 Buyer screens and related requirements. 28
22 Administration screens and related requirements. 38
23 Prototype scope and remaining work. 44

Scope of this document

This document specifies the requirements of a recommender-enabled electronic e-commerce platform developed as the laboratory platform for the course topics *Association Rules Mining*, *Recommender Systems* and *Word Embedding*. The e-commerce platform, the administration dashboard and the data layer constitute the operational part of the system. The intelligent component, a hybrid recommender that combines association rules learned from purchase transactions with semantic similarity learned from product descriptions, constitutes the research part, together with the data-warehouse pipeline that prepares the training data on which the recommender depends.

The specification answers three questions: *who* interacts with the system, *what* the system must do for them, and *how* these capabilities are organised into use cases. Section 1 describes the role and benefits of each stakeholder group. Section 2 lists the functional requirements with their priority, including the requirements of the data-warehouse pipeline that supplies training data to the models. Section 3 presents the general use-case model, the use-case catalogue and the main flows of the central use cases. Section 3.2 provides the requirement-to-use-case traceability mapping in Table 7, covering both requirements and catalogue entries. Section 4 introduces the selected dataset, describes its files and fields, and presents the adapted high-level operational data model. Section 5 presents the interactive prototype and interface mockups, distinguishing demonstrated user interactions from the remaining analytical implementation. Detailed warehouse schemas, programming interfaces and deployment procedures are documented separately in the project's architecture description. Warehouse-schema design and ETL authoring are project responsibilities carried out through external scripts and notebooks; the functional requirements describe execution of the resulting pipeline and the recommendation workflow supported by the system.

For this project, the model-development data consists of completed purchase transactions and available product text. Orders retain their items, quantities, purchase-time prices, timestamps and status; the catalogue retains product identifiers, categories and available names, descriptions and tags. Versioned datasets, training parameters, model artefacts and offline evaluation results make experiments repeatable. The current cart supplies recommendation context, but historical product views, clicks, cart actions and recommendation impressions are outside the recording scope. Purchases are retained as transactions without attribution to recommendations. Dataset refresh and model improvement are initiated manually by the data scientist.

1 Role and Benefits of Stakeholders

The application applies Association Rule Mining to purchase transactions to discover products that are frequently bought together, and Word Embeddings to product text to identify semantically related products. Three stakeholder groups interact with it: **product buyers**, **administrators** and **data scientists**. Buyers generate purchase transactions, data scientists consolidate them with catalogue data in a warehouse and turn them into models, and administrators operate the store, safeguard the input data and inspect the active model. Table 1 summarises the three groups.

Table 1: Summary of stakeholders, their touch-points and primary benefits.

Stakeholder	Main role	Touch-point	Primary benefit
Product Buyer	Browses, compares and purchases products; generates purchase transactions	Public storefront	Finds suitable products faster; receives explainable "bought together" and "similar product" suggestions
Administrator	Manages the catalogue, views orders, tracks sales, safeguards data quality, views the active model	Administration dashboard	Operates the store from a single interface; automated product suggestions; visibility of the active model
Data Scientist	Runs the warehouse pipeline, mines rules, trains embeddings, evaluates and publishes models, refreshes experiments and selects the active configuration	Analytics workspace (external notebooks and scripts, data warehouse)	A realistic case study with clean, versioned data and a measurable evaluation loop

1.1 Product Buyers

Product buyers are the end consumers of the storefront: they browse categories, search by keyword, inspect product details, maintain a cart and complete purchases, without registration. Their role is to select products and respond to the recommendations shown to them. Opening a product page triggers rule-based "frequently bought together" suggestions and embedding-based "similar product" suggestions; changing the cart recomputes them over the whole basket. Completed purchases are recorded as orders and line items with quantities, purchase-time prices, timestamps and status. These transactions, together with maintained catalogue text, provide data for offline training and evaluation alongside the imported historical purchases. Cart contents are persisted as current shopping state and used to request suggestions; they are not an interaction history. Product

views and recommendation displays or clicks are not recorded for model development.

Buyers benefit from data-driven suggestions instead of scanning the whole catalogue: association rules reveal complementary items that are easy to overlook, embeddings reveal substitutes when a product is unavailable or too expensive, and each suggestion carries a short explanation that builds trust. A persistent cart and a one-step checkout reduce the effort of completing an order.

1.2 Administrators

Administrators operate and maintain the store through the administration dashboard; they understand the retail business and need no machine-learning background. Their responsibilities are catalogue management, including withdrawing products without losing order history; viewing orders and their contained products; monitoring sales through revenue, order volume, average order value and best-sellers; and, in the extended system, overseeing the recommender in production by viewing which model version or configuration is active. Because the embedding model learns from available product names, descriptions, categories and tags, administrators are also indirectly responsible for the quality of that text.

Administrators benefit from a single, validated interface for all operations, from reports that remain correct when products are withdrawn, and from automated product suggestions without manually curating product pairs. Serving reports from the data warehouse keeps long analytical queries away from the storefront, and the modular design makes the system easier to maintain and extend.

1.3 Data Scientists

Data scientists, including the students carrying out the laboratory, are responsible for the analytical and experimental side of the application. As project responsibilities performed through external scripts and notebooks, they *design the data warehouse schema*: identify the purchase processes to analyse, fix the grain of each fact table (one purchased order line), define its measures and the conformed product, category and calendar-date dimensions, including surrogate keys and the handling of changing product attributes such as price and category, and author and modify the ETL pipeline. These development activities are outside the application's use-case boundary. Through the analytics workspace they execute and refresh the resulting pipeline, and extract versioned datasets from successful warehouse loads: distinct-product baskets from completed orders for rule mining, an available product-text corpus for embeddings, and chronologically separated purchases for offline evaluation.

On these datasets they mine association rules with Apriori or FP-Growth, tuning support, confidence and lift; train word embeddings with Word2Vec or FastText, or use a sentence-embedding model, to obtain one vector per product; and configure how association-based and embedding-based candidates are combined, with popularity as a fallback when learned candidates are insufficient. Rule-only, embedding-only and hybrid approaches are evaluated separately using suitable offline evidence. Purchase-basket completion evaluates rule-based and hybrid recommendations against a popularity baseline using precision, recall and catalogue coverage at a fixed list length on later held-out orders. Similar-product embeddings are assessed through a documented manual relevance review of a fixed sample against a category-based baseline, since substitutes need not be bought together. The data scientist stores the versioned artefacts and evaluation results, publishes versions that meet the acceptance criteria, and has sole responsibility for selecting and activating the model or configuration used by the storefront. Publishing makes an evaluated version available; selection determines which available version is served. Both actions may be performed consecutively by the same data scientist. When new completed purchases or catalogue changes are available, the data scientist manually refreshes the warehouse and datasets, reruns training and evaluation, and compares the new results before deciding whether to publish and activate a replacement.

Data scientists benefit from a realistic e-commerce case study in which transactional and textual data come from one live system, from a warehouse whose schema is shaped in advance for rule mining, embedding and evaluation, and from snapshot-based training that isolates experiments from operations and makes them repeatable and comparable. Stored dataset versions, training parameters, model versions and evaluation results support model improvement without an online interaction-feedback pipeline or claims about purchases caused by recommendations.

2 Functional Requirements

The following tables group functional requirements by stakeholder and pipeline. Each requirement has an identifier: FR-B for buyers, FR-A for administrators, FR-D for data scientists and FR-W for the warehouse pipeline. Table 7 maps these identifiers to the use-case catalogue. Schema design and ETL authoring remain the project responsibilities described in Section 1, rather than application functional requirements. Priority follows the MoSCoW scheme: **M**ust have, **S**hould have and **C**ould have.

2.1 Requirements for Product Buyers

Table 2: Functional requirements for product buyers.

Requirement	Priority
FR-B01 The buyer can browse, search and view products available on the e-commerce platform.	M
FR-B02 The buyer can add products to a shopping cart, update the cart and complete a purchase.	M
FR-B03 Completed purchases are stored as orders and line items retaining order and product identifiers, quantities, purchase-time prices, purchase timestamps and status. These records support purchase-basket mining, offline evaluation and sales reporting without recommendation attribution.	M
FR-B04 The buyer receives "frequently bought together" recommendations derived from association patterns learned from historical completed purchase transactions.	M
FR-B05 The buyer receives additional "similar product" recommendations derived from semantic similarity between product representations produced using word embeddings.	M
FR-B06 Recommendations are presented with a short explanation indicating why a product is suggested, such as being frequently associated with products in the current context or being semantically similar to the product being viewed.	S

2.2 Requirements for Administrators

Table 3: Functional requirements for administrators.

Requirement	Priority
FR-A01 The administrator can create, update and withdraw products, including their catalogue information such as name, category, price, stock level, description and tags. These operational changes are stored in the operational database.	M
FR-A02 The administrator can view orders and the products contained in each order.	M
FR-A03 The administrator can view analytical information derived from the data warehouse, including revenue, order volume, average order value, best-selling products and sales trends over time.	M
FR-A04 The administrator can view which recommendation model or recommendation configuration is currently active.	S

2.3 Requirements for Data Scientists

Table 4: Functional requirements for data scientists.

Requirement	Priority
FR-D01 The data scientist can manually execute and refresh the ETL pipeline that transforms operational product and purchase-transaction data into the analytical structures stored in the data warehouse.	M
FR-D02 The data scientist can obtain versioned completed-order baskets, available product-text corpora and chronological purchase hold-outs from the data warehouse for mining, semantic product representation and offline recommendation experiments.	M
FR-D03 The data scientist can apply Association Rule Mining to historical completed purchase baskets to discover product associations used for "frequently bought together" recommendations.	M
FR-D04 The data scientist can use word-embedding techniques to produce semantic representations of products from their textual information and identify semantically similar products.	M
FR-D05 The data scientist can combine association-rule-based candidates and embedding-based candidates into a hybrid recommendation strategy, while also retaining the ability to analyse the two recommendation approaches separately.	M
FR-D06 The data scientist can store versioned association rules, product representations, hybrid configuration and model metadata, including source dataset versions and training parameters, so that experiments can be reproduced and the artefacts evaluated, published and used by the platform.	M

Requirement	Priority
FR-D07 The data scientist can evaluate rule-based and hybrid basket-completion recommendations on later held-out purchases using precision, recall and catalogue coverage at a fixed list length against a popularity baseline. Embedding-based similar products are assessed through a documented manual relevance review of a fixed sample against a category-based baseline. Metrics and review results are stored with the corresponding model and dataset versions. The evaluation protocol and acceptance criteria are recorded before evaluating a candidate.	M
FR-D08 The data scientist can publish evaluated model versions that meet the defined acceptance criteria, releasing their association rules, product representations, hybrid configuration and model metadata so that they are available for selection and activation by the data scientist.	M
FR-D09 The data scientist can select and activate an available published recommendation model or configuration for use by the e-commerce platform, and view confirmation of the active selection. Model selection and activation are restricted to this role.	S
FR-D10 After new completed purchases or catalogue updates, the data scientist can manually refresh the datasets, retrain affected models and rerun offline evaluation. The resulting versions and results remain available for comparison before publication and activation.	S

2.4 Requirements for the Data-Warehouse Pipeline

The data warehouse supports both the intelligent recommendation workflow and the analytical functions of the administration dashboard. It consolidates operational data into a structure that can be used for Association Rule Mining, word-embedding-based recommendation and business statistics.

Table 5: Functional requirements for the data-warehouse pipeline.

Requirement	Priority
FR-W01 The ETL pipeline extracts relevant product, order and purchased-item data from the operational database and loads the transformed data into a separate analytical data warehouse.	M
FR-W02 The data warehouse contains fact and dimension structures that preserve purchased items, quantities, purchase-time prices, order status, product information and purchase timestamps required by the recommender system and administrative reports.	M

Requirement	Priority
FR-W03 The warehouse provides distinct-product baskets grouped by eligible completed order for Association Rule Mining. Quantities remain available for sales reporting; repeated units of the same product do not create duplicate basket entries.	M
FR-W04 The warehouse provides available product text, including categories and any maintained names, descriptions and tags, for embedding-based representations. The initial Olist dataset uses translated category text; any richer project catalogue text is documented separately.	M
FR-W05 The warehouse provides aggregated historical information required by the administration dashboard, including sales, orders and product performance over time.	M
FR-W06 The data scientist can manually rerun the pipeline after new purchase transactions or catalogue changes to refresh the warehouse and the datasets used by the recommendation system.	S

3 General Use-Case Model

Figure 1 shows the general use-case model. The three actors operate in three subsystems that are connected through data rather than through direct interaction: buyers create purchase transactions, the warehouse pipeline consolidates them with catalogue data, data scientists extract training datasets from the warehouse, and the model selected and activated by the data scientist supplies the recommendations shown to buyers. Administrators view orders and sales reports, maintain the catalogue text on which embeddings depend, and view the active model or configuration. The analytics workspace exposes pipeline execution and model operations; warehouse-schema design and ETL authoring take place in external development tools.

Solid lines associate actors with their use cases. Grey dashed arrows labelled $\ll$*include*$\gg$ denote behaviour performed within another use case, namely recommendation delivery on product pages and cart updates. Purple dashed arrows denote data dependencies across subsystem boundaries. They describe the flow of model, purchase-transaction and catalogue information, rather than the order in which use cases must execute. Previously prepared warehouse snapshots, datasets and evaluation results may be reused; their availability is a precondition for subsequent operations.

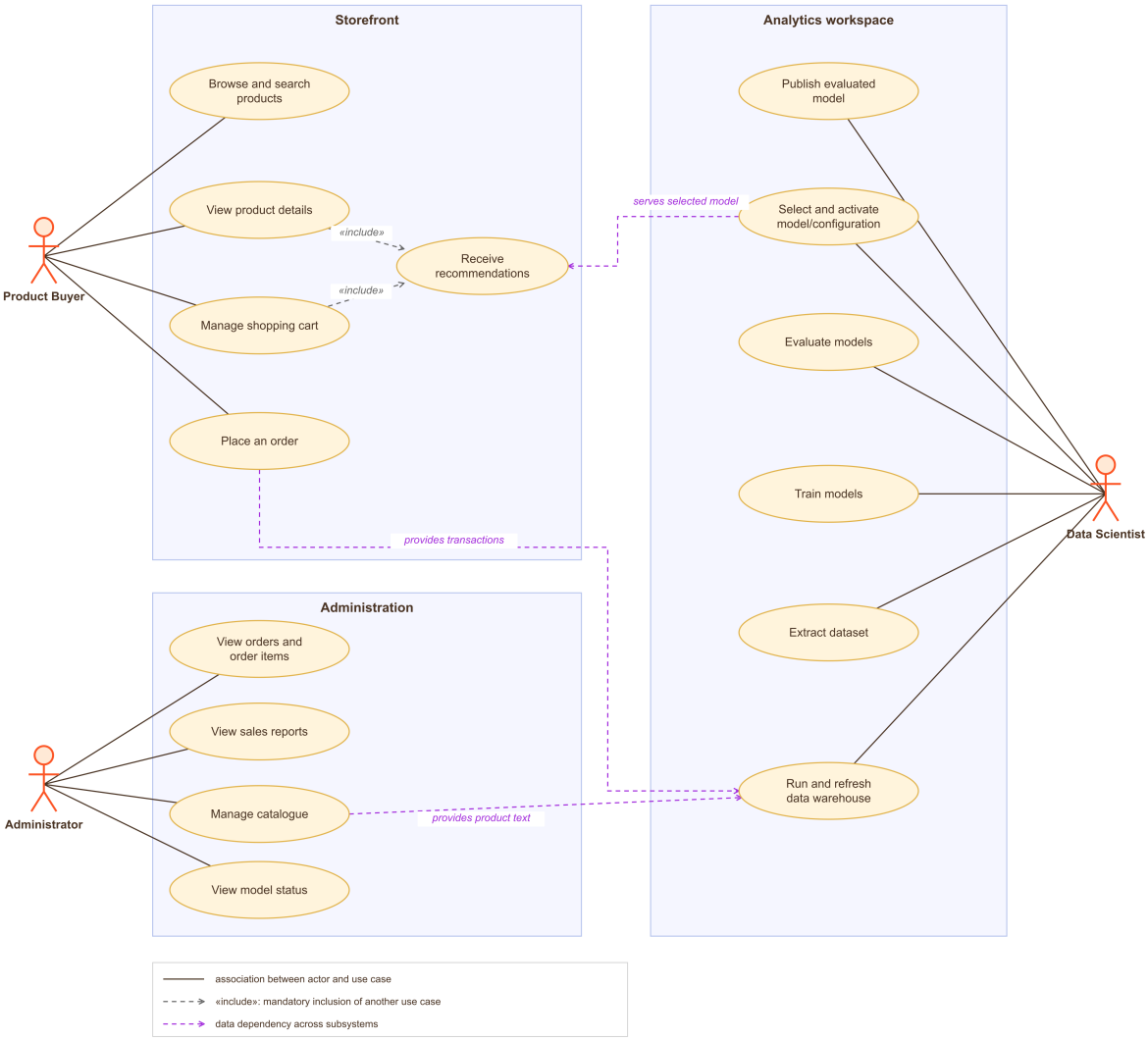


Figure 1: General use-case diagram of the recommender-enabled storefront.

3.1 Use-case catalogue

Table 6 summarises each use case with its primary actor and outcome.

Table 6: Use-case catalogue.

Use case	Actor	Summary	Outcome
Browse and search products	Buyer	List products by category; search, filter and sort	Matching products displayed
View product details	Buyer	Open a product page; includes *Receive recommendations*	Product attributes and recommendations displayed
Receive recommendations	Buyer (indirect)	Present complementary and similar suggestions for the current product or basket	Explained list generated using the active model and strategy
Manage shopping cart	Buyer	Add, update and remove items; includes *Receive recommendations*	Current cart updated and persisted; recommendations refreshed
Place an order	Buyer	Confirm the cart with minimal contact details	Order and line items recorded atomically; stock reduced; cart cleared
View orders and order items	Administrator	Inspect an order and its contained products	Order details and historical item information displayed
View sales reports	Administrator	Inspect warehouse-derived revenue, order volume, average order value, best-sellers and trends	Requested sales indicators and time series displayed
Manage catalogue	Administrator	Create, edit and withdraw products	Catalogue updated; withdrawn products kept in historical records
View model status	Administrator	Inspect the active recommendation model or configuration	Active version and configuration displayed
Run and refresh data warehouse	Data Scientist	Execute ETL into the established dimensional schema	Facts, dimensions, dataset views and reporting aggregates refreshed; load metadata and quality report recorded
Extract dataset	Data Scientist	Derive mining, product-text and evaluation data from a successful warehouse snapshot	Versioned datasets stored with lineage

Use case	Actor	Summary	Outcome
Train models	Data Scientist	Mine rules, train embeddings and configure the hybrid using available versioned datasets	Versioned rules, vectors, hybrid configuration and metadata stored; separate methods remain available
Evaluate models	Data Scientist	Assess basket completion on held-out purchases and review similar-product relevance against suitable baselines	Offline metrics and manual review results stored with model and dataset versions
Publish evaluated model	Data Scientist	Release stored rules, vectors, hybrid configuration and metadata after checking acceptable evaluation results	Published version available for selection and activation
Select and activate model/configuration	Data Scientist	Select an available published version or configuration for online serving	Active selection updated and confirmed; subsequent recommendations use it

3.2 Requirement-to-use-case traceability

Table 7 links every functional requirement to the use case or use cases that describe its behaviour. A use case may cover several requirements, and a requirement may involve several use cases. Purchase persistence is part of placing an order; warehouse refresh and model experiments are separate Data Scientist operations. Schema design and ETL authoring are outside this mapping because they are project responsibilities rather than application functional requirements. Every catalogue entry appears in the mapping, so it can also be read from a use case to the requirements it fulfils.

Table 7: Functional requirement-to-use-case traceability.

Requirement ID	Corresponding use case(s)
FR-B01	Browse and search products; View product details
FR-B02	Manage shopping cart; Place an order
FR-B03	Place an order
FR-B04	Receive recommendations
FR-B05	Receive recommendations
FR-B06	Receive recommendations

Requirement ID	Corresponding use case(s)
FR-A01	Manage catalogue
FR-A02	View orders and order items
FR-A03	View sales reports
FR-A04	View model status
FR-D01	Run and refresh data warehouse
FR-D02	Extract dataset
FR-D03	Train models
FR-D04	Train models
FR-D05	Train models; Evaluate models
FR-D06	Train models; Publish evaluated model
FR-D07	Evaluate models
FR-D08	Publish evaluated model
FR-D09	Select and activate model/configuration
FR-D10	Run and refresh data warehouse; Extract dataset; Train models; Evaluate models
FR-W01	Run and refresh data warehouse
FR-W02	Run and refresh data warehouse
FR-W03	Run and refresh data warehouse; Extract dataset
FR-W04	Run and refresh data warehouse; Extract dataset
FR-W05	Run and refresh data warehouse; View sales reports
FR-W06	Run and refresh data warehouse; Extract dataset

3.3 Main flows of the central use cases

Receive recommendations. The storefront requests suggestions for the current product or cart contents. Using the active model and configuration, the recommendation service retrieves complementary candidates from matching association rules and similar-product candidates from product embeddings. The hybrid configuration determines their ranking and combination; popular products fill remaining gaps when learned candidates

are insufficient. Each suggestion carries its model version, strategy and a short reason. The storefront displays the suggestions with their reasons. Product-page and cart changes request updated suggestions using the current context; they do not create historical interaction records.

Place an order. The buyer confirms the cart with minimal contact details. The order service validates the request and checks availability, then records the order, its line items with the price at the time of purchase, and the stock adjustments in one atomic operation. The order retains its purchase timestamp and status, and the buyer receives an order number. Completed orders become the durable transaction history used by the warehouse; no purchase-to-recommendation attribution is recorded. If any item is insufficiently stocked, the whole operation is cancelled and the affected product is reported.

View orders and order items. The administrator selects an order from the operational order records. The system displays its details and contained products, including quantities and prices recorded at purchase time. Historical order information remains accessible when a catalogue product is withdrawn.

View sales reports. The administrator chooses the reporting period and inspects warehouse-derived revenue, order volume, average order value, best-selling products and sales trends. Reports reflect the latest successful warehouse refresh and retained purchase-time quantities and prices.

View model status. The administrator inspects the currently active model version and recommendation configuration from the serving state. This visibility is separate from sales reporting and does not grant model selection or activation rights.

Run and refresh data warehouse. When manually requested by the data scientist, the pipeline extracts orders, purchased items and products changed since the last successful load. It transforms them to the dimensional schema (surrogate keys, conformed dimensions and facts at purchased-order-line grain), validates integrity, completeness, duplicates and value ranges, and loads the result into the schema prepared during project development. Failed checks reject the load; successful loads refresh training-dataset views and sales aggregates used by the administration dashboard, and record their lineage. Subsequent runs process new purchase transactions and catalogue changes. Executing ETL is separate from designing or modifying its implementation.

Extract dataset. A successful warehouse snapshot is a precondition. The data scientist extracts distinct-product baskets from eligible completed orders, available product text

including categories and any maintained names, descriptions and tags, and chronological purchase data for evaluation. Imported Olist records initially supply purchase baskets and translated category text; richer product text, if added, comes from separately documented catalogue enrichment. Quantities are obtained from stored order-item quantities or counted source item rows as appropriate. Each dataset is stored with its source snapshot, scope and version. Extraction can reuse an existing warehouse snapshot without rerunning ETL.

Model life cycle. Training requires available versioned datasets. The data scientist mines rules with chosen support, confidence and lift thresholds, produces one embedding vector per product, and configures the hybrid combination and fallback behaviour. Rules, vectors, configuration and metadata are stored as versioned artefacts, retaining separately usable association-based and embedding-based approaches. Training experiments may reuse the same datasets without extracting them again.

Evaluation requires stored model artefacts and suitable versioned evaluation data. Before running evaluation, the data scientist records the experiment protocol and acceptance criteria: the dataset scope and split, list length, baseline, required metric thresholds for basket completion, and the sample, relevance rubric and minimum acceptable result for the similar-product review. These experiment settings are fixed before inspecting the candidate's evaluation results and retained with the model and dataset versions. Publication checks the stored results against those criteria; missing results or unmet criteria prevent publication. The thresholds are experiment parameters, rather than evidence of online recommendation effectiveness.

Earlier purchases train the models; later eligible multi-product orders form a chronological hold-out. Basket-completion experiments expose part of each held-out basket as context and use its remaining products as targets. Rule-only and hybrid recommendations are compared with a popularity baseline learned only from training data, using precision and recall at a fixed list length and catalogue coverage across the evaluated recommendation lists. The split, context/target procedure and list length are recorded.

Embedding-based similar products are assessed separately using a fixed sample of product queries and a documented manual relevance rubric, with a category-based baseline evaluated on the same queries. The review records the catalogue-text source, sample, relevance judgements and model version; category-only Olist text limits the specificity of this initial semantic evaluation. Purchase co-occurrence does not establish whether two products are suitable substitutes. The experiment results support offline model comparison, without measuring recommendation-attributed sales.

Results are stored with the model and dataset versions. Publication requires acceptable

stored evaluation results under the documented acceptance criteria for the exact version being released; it makes that version's rules, vectors, configuration and metadata available without rerunning evaluation or automatically changing the active selection.

Refresh model experiments. When new completed purchases or catalogue updates become available, the data scientist manually runs the warehouse refresh, extracts new versioned datasets and repeats the affected training and evaluation experiments. New results are compared with the previous version before deciding whether to publish and activate a replacement. Training-parameter trials may reuse an existing dataset. This workflow reuses the warehouse, extraction, training and evaluation use cases; it does not require interaction-event collection or automatic retraining.

Select and activate model/configuration. The data scientist chooses an available published model version or configuration. The system checks that the selected artefacts are available, activates the selection for online serving and confirms the active version/configuration. Subsequent recommendation requests use that selection, which the administrator can inspect through *View model status*. If activation fails, the current active selection remains in use. Publication and activation may be performed consecutively by the data scientist, who alone controls both operations.

4 Dataset Description and Project Integration

4.1 Dataset overview

The project will use the *Brazilian E-Commerce Public Dataset by Olist*, available on Kaggle [1]. Olist released anonymised commercial records for approximately 100,000 orders placed between 2016 and 2018 across multiple marketplaces in Brazil. The data describes customers, products, sellers, orders, purchased items, payments, delivery information and customer reviews. It covers multiple product categories, providing a historical basis for the project's operational database, sales analytics and association-rule experiments.

The release consists of nine related CSV files. They form a relational data model: orders connect customers to purchased items, and each item connects an order to a product and a seller. Payments and reviews describe the same orders, while geolocation and category translations provide additional reference information. This connected structure allows the project to reconstruct purchase baskets and prepare analytical datasets through the ETL pipeline.

4.2 Files and fields

The following tables give a simple description of all 52 fields across the nine files. Attribute names follow the original CSV headers. The data types describe how values will be represented after import; CSV files themselves do not declare database types. String denotes text, Integer denotes a whole number, Decimal denotes a numeric value with a fractional part, and Timestamp denotes a date and time. Identifiers and postal-code prefixes will be stored as strings to preserve their original form, including leading zeros. Missing values will be checked during import rather than treated as zero.

4.2.1 Customers

File: `olist_customers_dataset.csv`.

Customer identifiers and delivery locations. The same person can have different customer IDs across orders; the unique customer ID links repeat purchases.

Table 8: Fields in the customers file.

Attribute	Data type	Description
`customer_id`	String	Customer identifier used to link a customer record to an order.
`customer_unique_id`	String	Identifier for the same customer across different orders.

Attribute	Data type	Description
customer_zip_code_prefix	String	First five digits of the customer postal code.
customer_city	String	City of the customer delivery address.
customer_state	String	Brazilian state of the customer delivery address.

4.2.2 Orders

File: olist_orders_dataset.csv.

Order headers, including the customer, order status and purchase and delivery dates.

Table 9: Fields in the orders file.

Attribute	Data type	Description
order_id	String	Identifier of the order.
customer_id	String	Links the order to its customer record.
order_status	String	Status of the order, such as delivered or canceled.
order_purchase_timestamp	Timestamp	Date and time when the order was placed.
order_approved_at	Timestamp	Date and time when the order payment was approved.
order_delivered_carrier_date	Timestamp	Date and time when the order was handed to the carrier.
order_delivered_customer_date	Timestamp	Date and time when the customer received the order.
order_estimated_delivery_date	Timestamp	Delivery date estimated at the time of purchase.

4.2.3 Order items

File: olist_order_items_dataset.csv.

Items purchased within each order. An order can contain several items, possibly supplied by different sellers. An item is identified by the combination of order ID and item sequence.

Table 10: Fields in the order items file.

Attribute	Data type	Description
`order_id`	String	Links the item to its order.
`order_item_id`	Integer	Sequential item number within the order; not a quantity field.
`product_id`	String	Links the purchased item to its product.
`seller_id`	String	Links the purchased item to the seller fulfilling it.
`shipping_limit_date`	Timestamp	Deadline for the seller to hand the item to the logistics partner.
`price`	Decimal	Sale price of the item, excluding its freight charge.
`freight_value`	Decimal	Freight charge associated with the item.

4.2.4 Order payments

File: `olist_order_payments_dataset.csv`.

Payment records for orders. An order may use more than one payment method, so several payment records can belong to the same order.

Table 11: Fields in the order payments file.

Attribute	Data type	Description
`order_id`	String	Links the payment record to its order.
`payment_sequential`	Integer	Sequence distinguishing payment records within an order.
`payment_type`	String	Payment method, such as credit card, voucher or boleto.
`payment_installments`	Integer	Number of installments selected for the payment.
`payment_value`	Decimal	Amount recorded for this payment.

4.2.5 Products

File: `olist_products_dataset.csv`.

Product categories and physical attributes. The name and description length fields contain character counts, rather than the original product text. The spelling `lenght` is retained from the source.

Table 12: Fields in the products file.

Attribute	Data type	Description
`product_id`	String	Identifier of the product.
`product_category_name`	String	Product category name in Portuguese.
`product_name_lenght`	Integer	Number of characters in the product name; the name itself is absent.
`product_description_lenght`	Integer	Number of characters in the description; the description itself is absent.
`product_photos_qty`	Integer	Number of photographs associated with the product.
`product_weight_g`	Integer	Product weight in grams.
`product_length_cm`	Integer	Product length in centimetres.
`product_height_cm`	Integer	Product height in centimetres.
`product_width_cm`	Integer	Product width in centimetres.

4.2.6 Sellers

File: `olist_sellers_dataset.csv`.

Identifiers and locations of sellers who fulfil the purchased items.

Table 13: Fields in the sellers file.

Attribute	Data type	Description
`seller_id`	String	Identifier of the seller.
`seller_zip_code_prefix`	String	First five digits of the seller postal code.
`seller_city`	String	City where the seller is located.
`seller_state`	String	Brazilian state where the seller is located.

4.2.7 Order reviews

File: `olist_order_reviews_dataset.csv`.

Customer satisfaction survey responses about the purchase experience. Reviews are linked to orders, so an order review is not automatically a rating of each individual product.

Table 14: Fields in the order reviews file.

Attribute	Data type	Description
`review_id`	String	Identifier attached to the review record.
`order_id`	String	Links the review to its order.
`review_score`	Integer	Customer satisfaction score from 1 to 5.
`review_comment_title`	String	Optional title written by the customer.
`review_comment_message`	String	Optional review comment written by the customer.
`review_creation_date`	Timestamp	Date when the satisfaction survey was sent to the customer.
`review_answer_timestamp`	Timestamp	Date and time when the customer submitted the survey response.

4.2.8 Geolocation

File: `olist_geolocation_dataset.csv`.

Geographic coordinates associated with Brazilian postal-code prefixes. Several location records can share the same prefix.

Table 15: Fields in the geolocation file.

Attribute	Data type	Description
`geolocation_zip_code_prefix`	String	First five digits of the postal code.
`geolocation_lat`	Decimal	Latitude of the location.
`geolocation_lng`	Decimal	Longitude of the location.
`geolocation_city`	String	City associated with the location.
`geolocation_state`	String	Brazilian state associated with the location.

4.2.9 Category translations

File: `product_category_name_translation.csv`.

Portuguese-to-English translations of product category names. Categories without a matching translation will retain their original label.

Table 16: Fields in the category translations file.

Attribute	Data type	Description
`product_category_name`	String	Portuguese category name used in the products file.
`product_category_name_english`	String	Corresponding English category name.

4.3 Project data model

Figure 2 shows the adapted relational model for the project. It separates a customer's persistent identity from the location snapshot associated with an order, connects products to categories and sellers, and links orders to their items, payments and reviews. The model retains purchase transactions and catalogue information; it does not include an interaction-event entity. Current cart state and experiment metadata are operational or analytical structures outside this identifier-focused diagram.

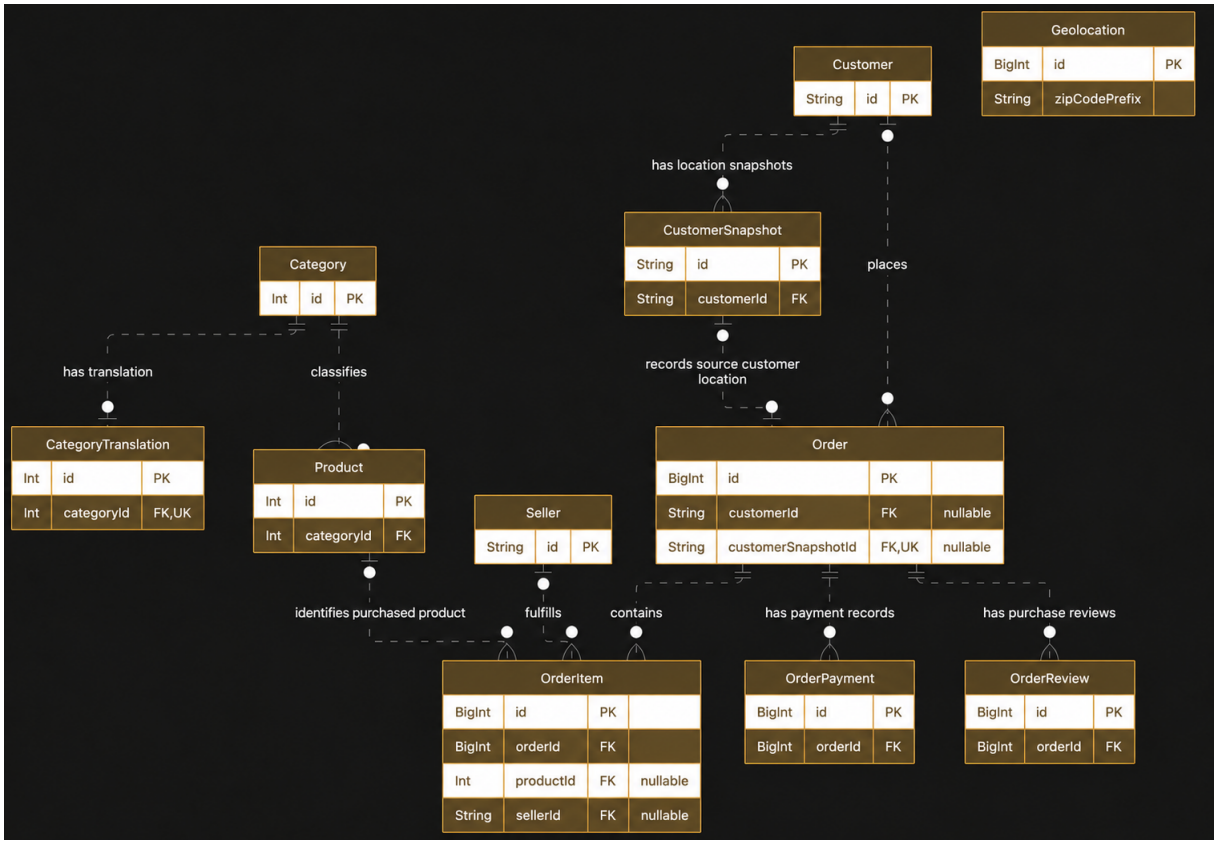


Figure 2: Adapted project data model and its entity relationships.

The following four tables describe the key attributes displayed in the diagram. Attribute names are qualified by entity, for example `Order.id`. String is a text identifier, Int is an integer, and Bigint is a larger integer used for generated record identifiers. PK denotes a primary key, FK a foreign key, and UK a uniqueness constraint. Nullable means that the relationship can be absent. The diagram focuses on identifiers and relationships; the descriptive, transactional and timestamp fields listed in the CSV tables will also be mapped to their corresponding project entities.

The source and project identifiers are distinct. Generated Int and Bigint keys will identify records inside the project, while the original Olist identifiers will be retained through source fields or mapping records. In particular, `customer_unique_id` will map to `Customer.id`, and the order-specific `customer_id` will map to `CustomerSnapshot.id`. This preserves repeat-customer identity and the customer location recorded for each historical order.

Customer identity and location.

Table 17: Project model attributes: customer identity and location.

Attribute	Data type	Description
`Customer.id`	String	PK. Persistent customer identifier, mapped from the source customer unique ID.
`CustomerSnapshot.id`	String	PK. Identifies the customer location snapshot associated with a source order.
`CustomerSnapshot.customerId`	String	FK to Customer.id. Links the snapshot to the persistent customer identity.
`Geolocation.id`	Bigint	PK. Generated identifier for a geographic reference record.
`Geolocation.zipCodePrefix`	String	Postal-code prefix used to match customer and seller locations to geographic records.

Product catalogue and categories.

Table 18: Project model attributes: product catalogue and categories.

Attribute	Data type	Description
`Category.id`	Int	PK. Generated identifier for a product category.

Attribute	Data type	Description
`CategoryTranslation.id`	Int	PK. Generated identifier for a category translation record.
`CategoryTranslation.categoryId`	Int	FK to Category.id and UK. Allows at most one translation record per category in this model.
`Product.id`	Int	PK. Generated product identifier, mapped to the original Olist product ID.
`Product.categoryId`	Int	FK to Category.id. Identifies the category classifying the product.

Orders, purchased items and sellers.

Table 19: Project model attributes: orders, purchased items and sellers.

Attribute	Data type	Description
`Seller.id`	String	PK. Seller identifier retained from the source seller ID.
`Order.id`	Bigint	PK. Generated order identifier, mapped to the original Olist order ID.
`Order.customerId`	String	Nullable FK to Customer.id. Identifies the buyer when a persistent customer is available.
`Order.customerSnapshotId`	String	Nullable FK to CustomerSnapshot.id and UK. Associates an order with its own customer location snapshot.
`OrderItem.id`	Bigint	PK. Generated item-record identifier; the source order ID and item sequence are retained for mapping.
`OrderItem.orderId`	Bigint	FK to Order.id. Identifies the order containing the purchased item.
`OrderItem.productId`	Int	Nullable FK to Product.id. Identifies the purchased product when a linked catalogue record is available.
`OrderItem.sellerId`	String	Nullable FK to Seller.id. Identifies the seller fulfilling the item when available.

Payments and reviews.

Table 20: Project model attributes: payments and reviews.

Attribute	Data type	Description
`OrderPayment.id`	Bigint	PK. Generated identifier for a payment record; its source order and payment sequence are retained.
`OrderPayment.orderId`	Bigint	FK to Order.id. Links a payment record to the order it pays for.
`OrderReview.id`	Bigint	PK. Generated review-record identifier, with the original review identifier retained for traceability.
`OrderReview.orderId`	Bigint	FK to Order.id. Links a satisfaction review to its purchase order.

Geolocation remains a reference entity in the supplied diagram; no direct geolocation foreign key is shown. Its postal-code prefix will be used for the location matching and consolidation described in the integration process below.

4.4 Adapting the data model to the project

We will adapt the Olist data model to the project's operational database and data warehouse. The CSV files will first be imported into staging tables so that the original records and identifiers remain available. Their entities will then be mapped to the project's customers, products, categories, sellers, orders, order items, payments and reviews. Geolocation and category translations will serve as reference tables. Original identifiers will be retained for traceability, while warehouse fact and dimension tables will use the keys and grain defined by the project's analytical schema.

The main links will be preserved: `customer_id` connects orders to customer records; `order_id` connects orders to items, payments and reviews; and `product_id` and `seller_id` connect items to products and sellers. `customer_unique_id` will identify repeat buyers. Category names will link products to translations, and postal-code prefixes will link customer and seller locations to geography. Location records will be consolidated by prefix before joining, and payment and review records will be handled at their own grain to avoid multiplying order-item values in reports.

For Association Rule Mining, the project will group distinct product IDs within eligible completed orders to construct purchase baskets. Purchase timestamps will support chronological training and evaluation. Historical item prices and freight charges will remain attached to their order items, rather than being overwritten by current catalogue

prices. The administration dashboard will use the resulting warehouse records to calculate sales trends, order counts and product performance. For the electronics storefront, a documented subset of relevant categories will be selected while retaining the source order links and recording the scope of each experimental dataset.

The source requires several adaptations to support the application. It does not provide original product names or descriptions or inventory balances. Translated category text will support an initial, limited semantic-recommendation baseline; richer catalogue text, if added for the project, will be documented separately and linked to the same product IDs. Demonstration stock levels and storefront display labels will be identified as project-created data.

New completed storefront purchases retain their order links, product identifiers, quantities, purchase-time prices, timestamps and status. Together with catalogue updates, they can be loaded into the warehouse alongside historical purchases during a manually initiated refresh. The data scientist then creates a new dataset version and retrains and evaluates affected models before publication or activation. Carts remain current shopping state; browsing, click, cart-action and recommendation-impression histories are outside this project's data model. This approach fits the dataset's relational structure into the project while distinguishing imported records, derived information and data collected or created by the new system.

5 Interactive Prototype and Interface Mockups

Signal Shop demonstrates the buyer, administrator and data-scientist interfaces for Section 2. The figures use report-preview mode with a curated electronics catalogue and sample orders. Recommendation pairings are hardcoded, similar products use category matches, and model values are illustrative.

5.1 Buyer interface and explained recommendations

The buyer flow is *browse or search* → *product details* → *recommendations* → *bag* → *checkout* → *confirmation*. Table 21 maps these screens to Table 2.

Table 21: Buyer screens and related requirements.

Screen	Information and actions	Requirements
Storefront and catalogue	Featured products, category browsing and sorting (Figures 3–4).	FR-B01
Search and details	Keyword results, product attributes, price and stock (Figures 5–6).	FR-B01
Bought together	Four complementary items with explanations on the product page (Figure 7).	FR-B04, FR-B06
Similar products	Four alternatives below the selected product, retaining search context (Figure 8).	FR-B05, FR-B06
Bag	Quantities, subtotal and suggestions that exclude selected products (Figure 9).	FR-B02, FR-B04, FR-B06
Checkout and confirmation	Minimal contact details and a simulated order number (Figures 10–11).	FR-B02

5.1.1 Storefront and featured products

Related requirements: FR-B01.

Figure 3 shows the main page and its featured product list. Buyers can open an item or continue to the full collection.

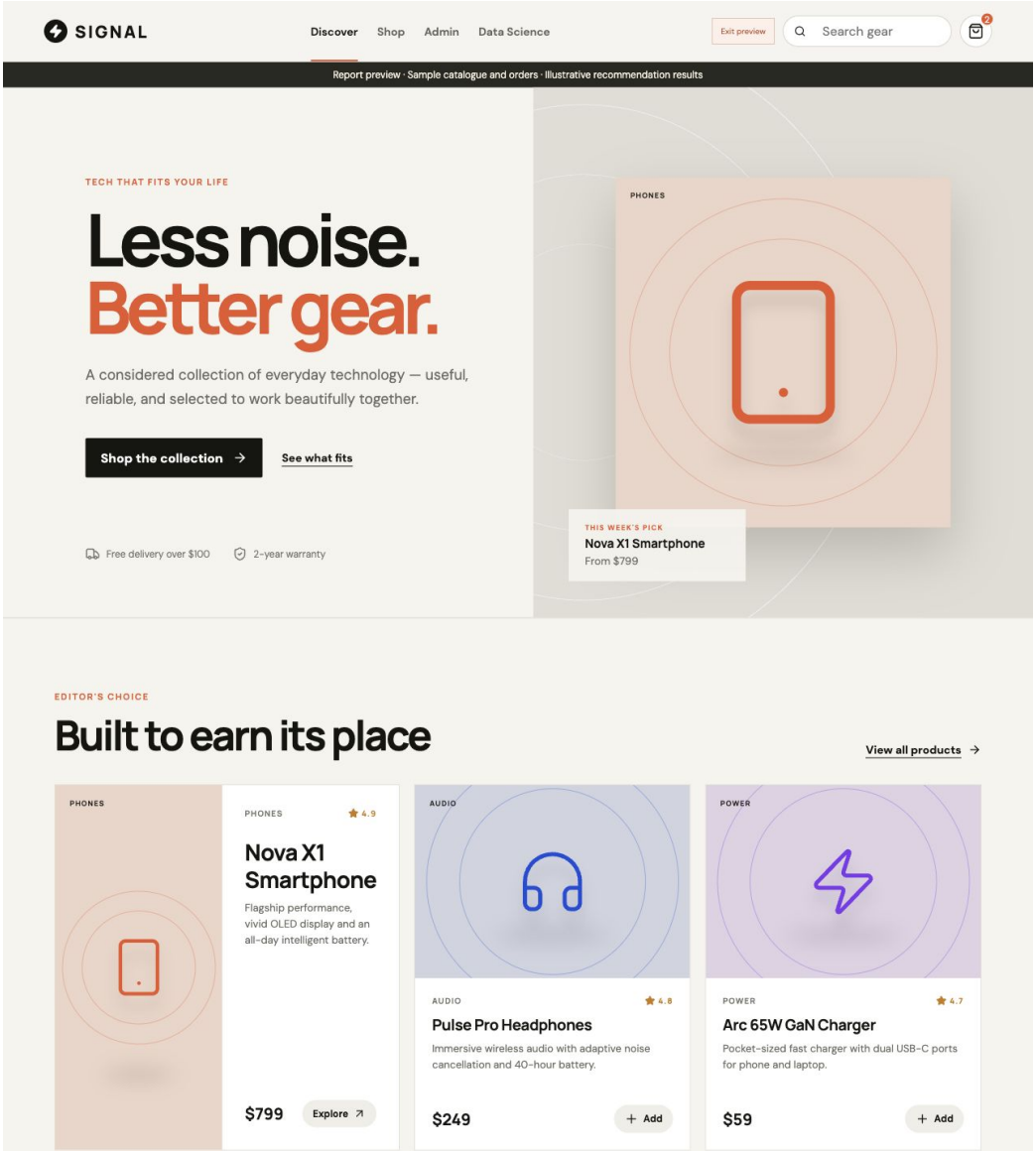


Figure 3: Main storefront with featured products and access to the collection.

5.1.2 Product catalogue

Related requirements: FR-B01.

Figure 4 shows the collection page with product cards, category filters and sorting. Each card provides a route to product details.

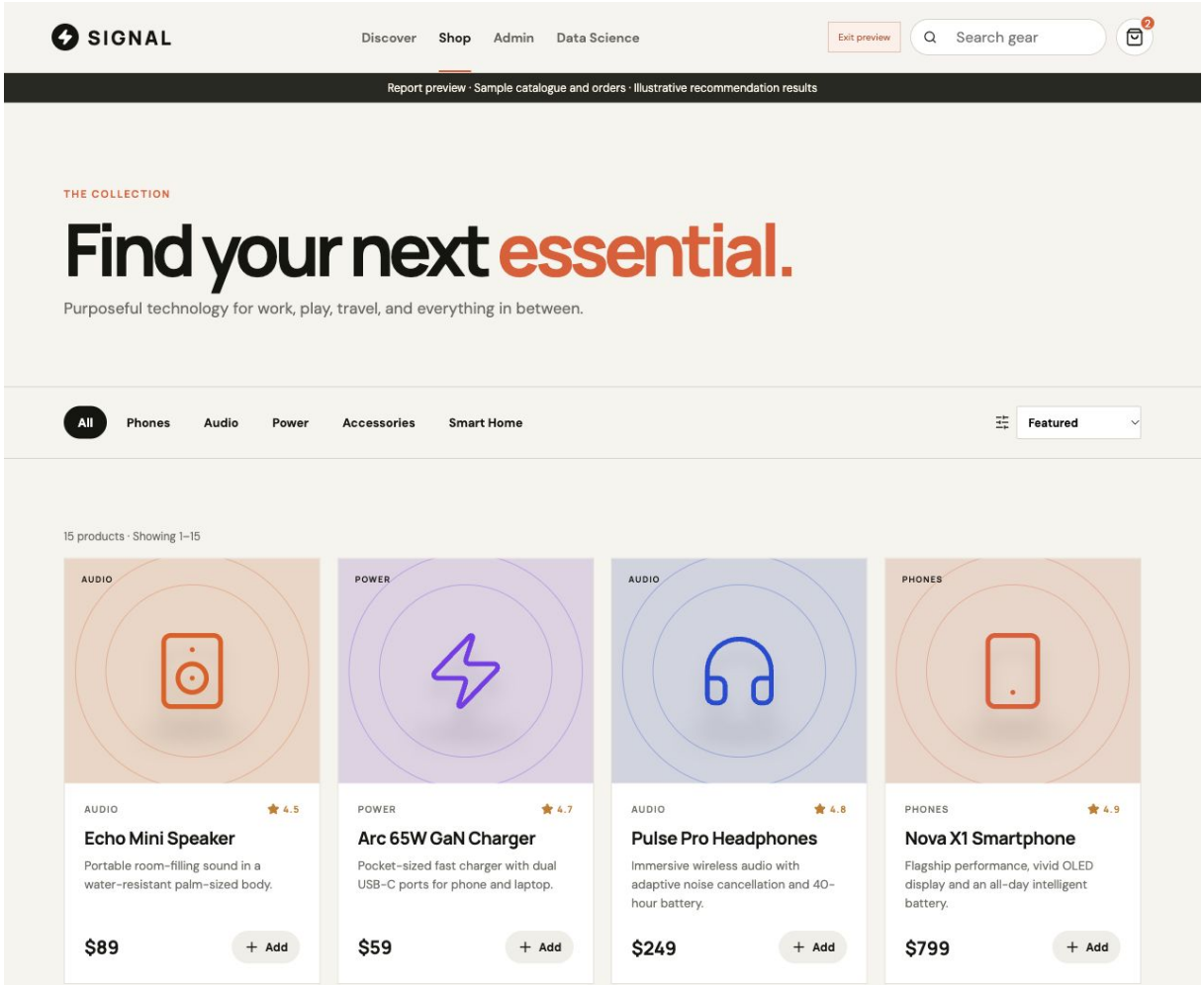


Figure 4: Product catalogue with category and sorting controls.

5.1.3 Search results

Related requirements: FR-B01.

Searching for “smartphone” returns matching products (Figure 5). The buyer selects Nova X1 to inspect its details and recommendations.

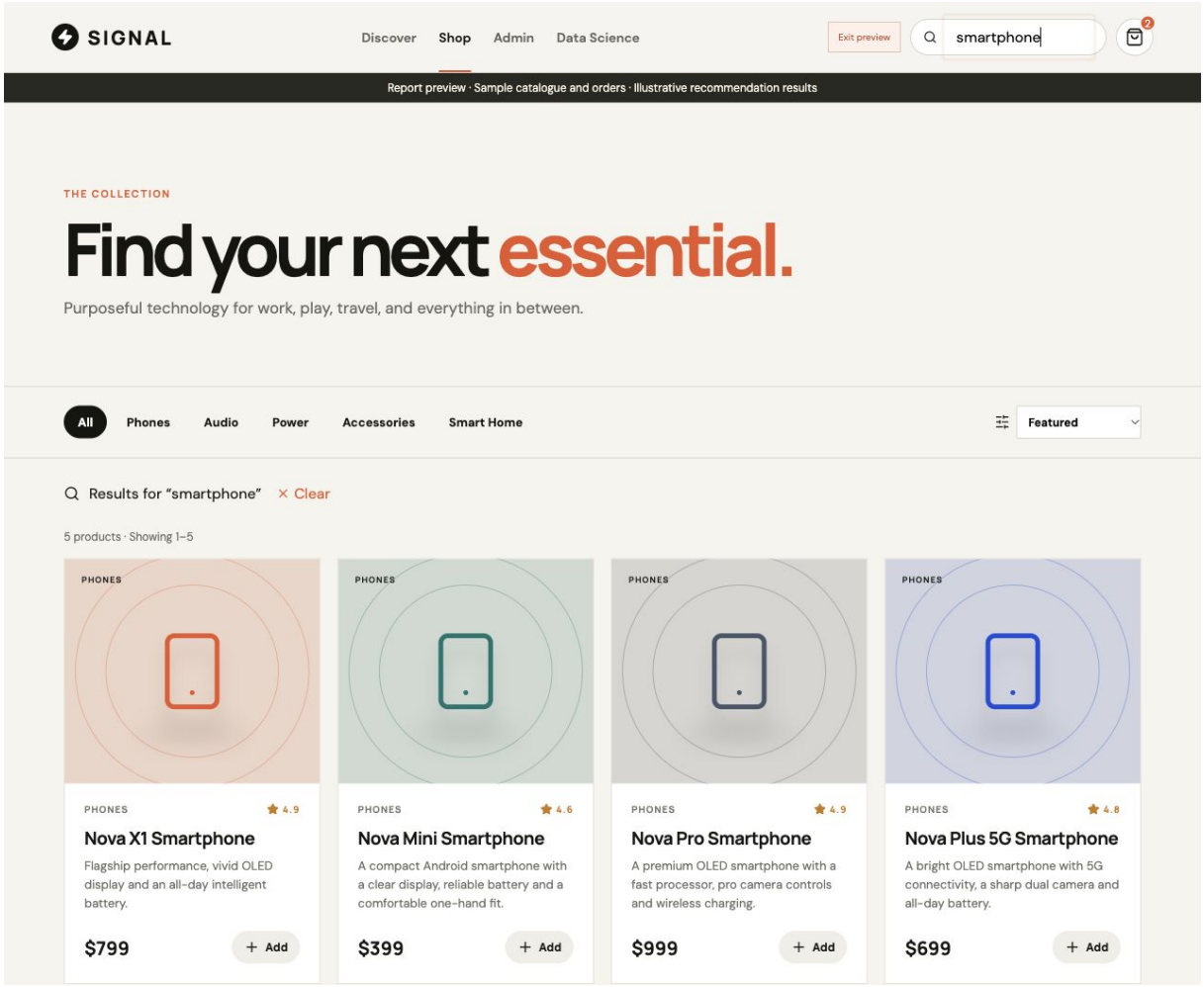


Figure 5: Keyword search results for smartphones.

5.1.4 Product details

Related requirements: FR-B01, FR-B02.

Figure 6 shows Nova X1's description, tags, price and stock. The search breadcrumb preserves the buyer's location, and *Add to bag* starts the purchase flow.

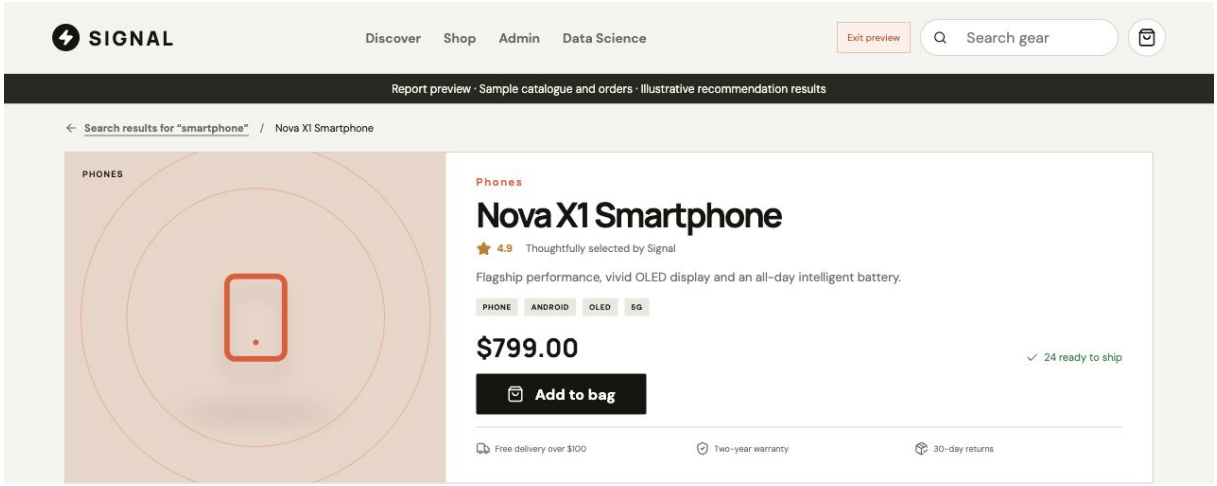


Figure 6: Selected smartphone with product information and an add-to-bag action.

5.1.5 Frequently bought together

Related requirements: FR-B04, FR-B06.

On the product page, selecting *Frequently bought together* reveals four complementary items (Figure 7). The selected product remains visible above the suggestions and their explanations.

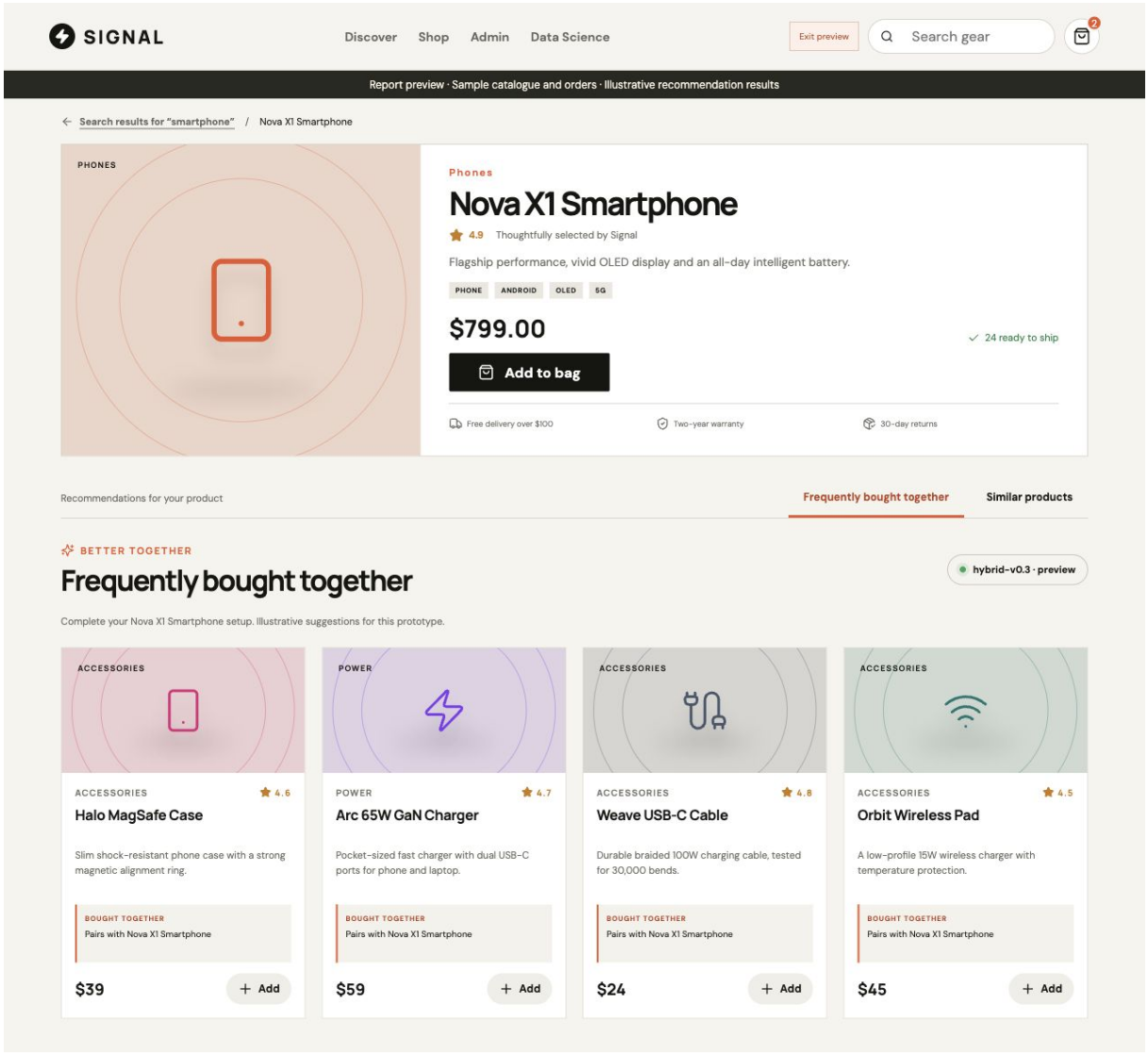


Figure 7: Nova X1 product page with explained complementary suggestions.

5.1.6 Similar products

Related requirements: FR-B05, FR-B06.

Selecting *Similar products* displays four smartphone alternatives (Figure 8). Product details, the search breadcrumb, prices and explanations make the suggestion context explicit.

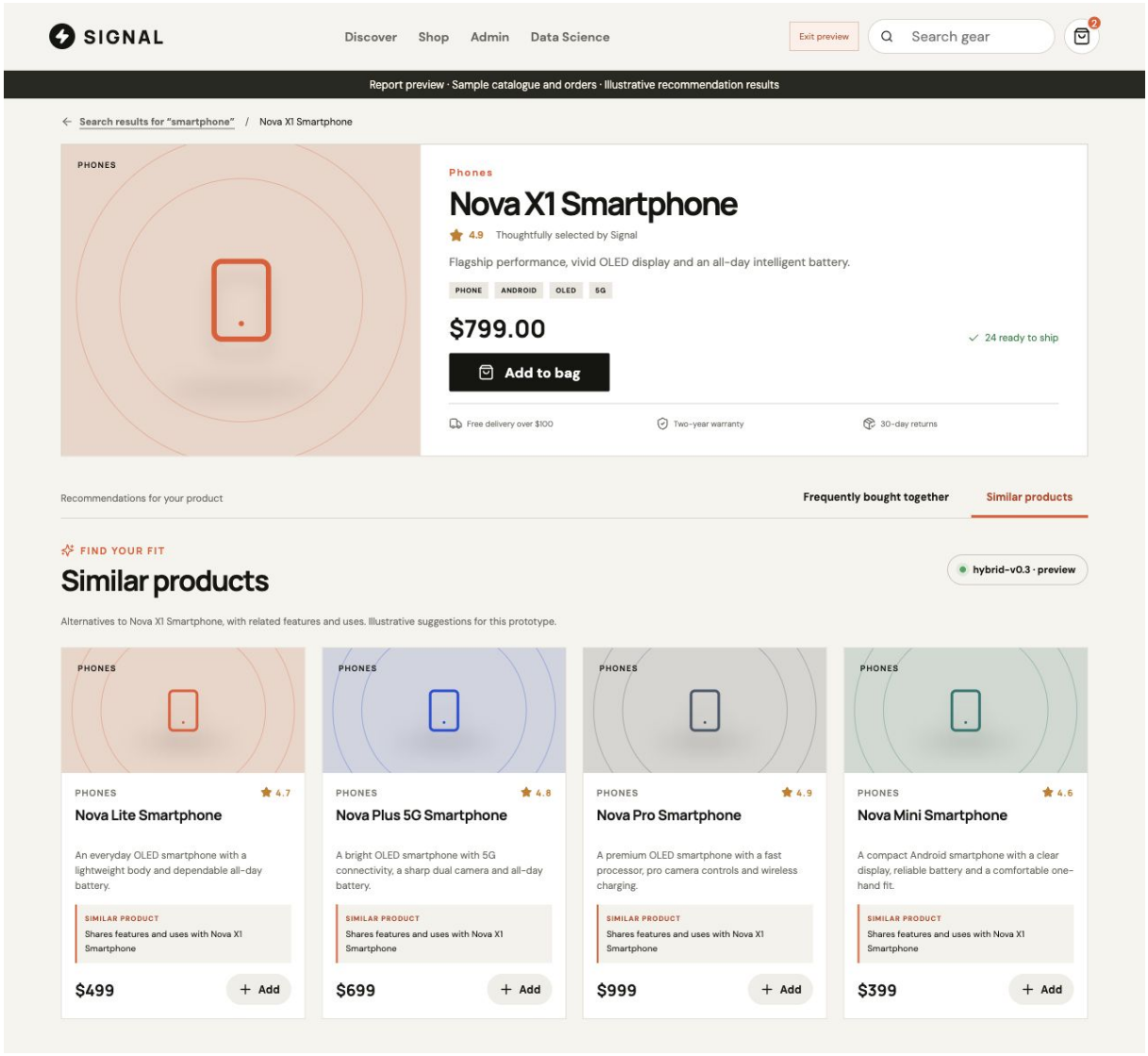


Figure 8: Nova X1 product page with four similar smartphone alternatives.

5.1.7 Shopping cart and basket recommendations

Related requirements: FR-B02, FR-B04, FR-B06.

Figure 9 shows the selected phone and charger, quantity controls and subtotal. Suggestions use the whole basket and exclude selected products; the checkout action continues the flow.

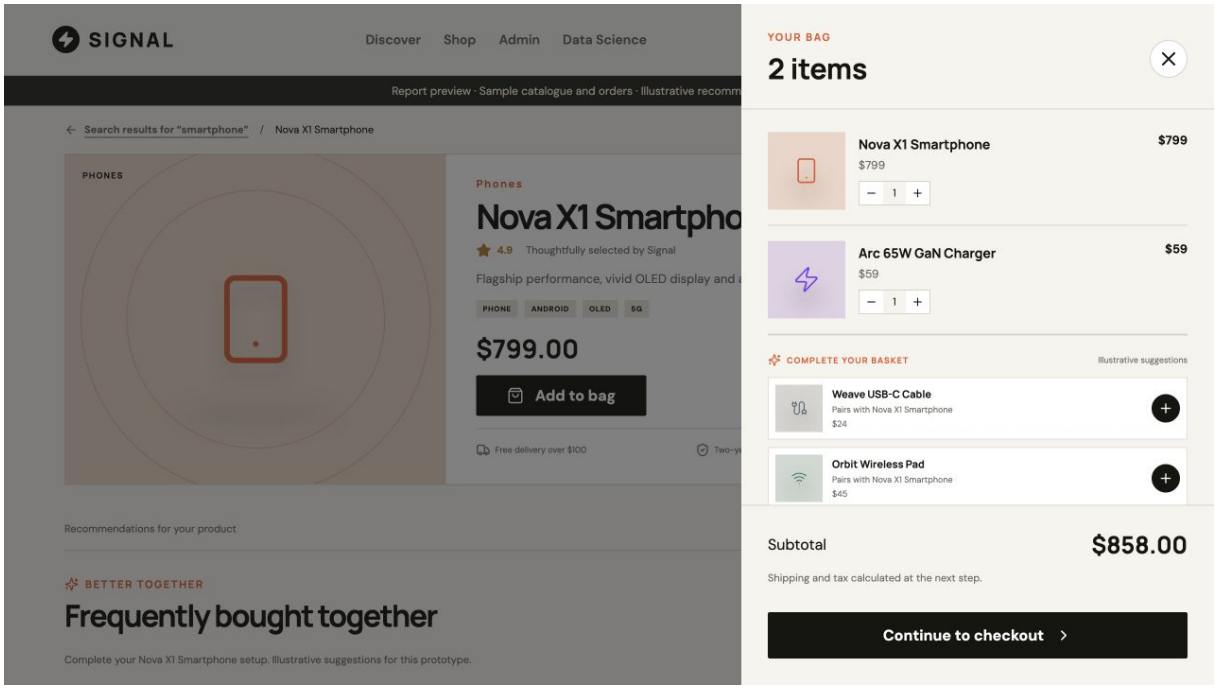


Figure 9: Shopping bag with selected products, subtotal and basket suggestions.

5.1.8 Checkout

Related requirements: FR-B02.

Figure 10 collects the buyer's name and email and shows the order total. The preview explicitly states that no payment is collected and the sample order is not saved.

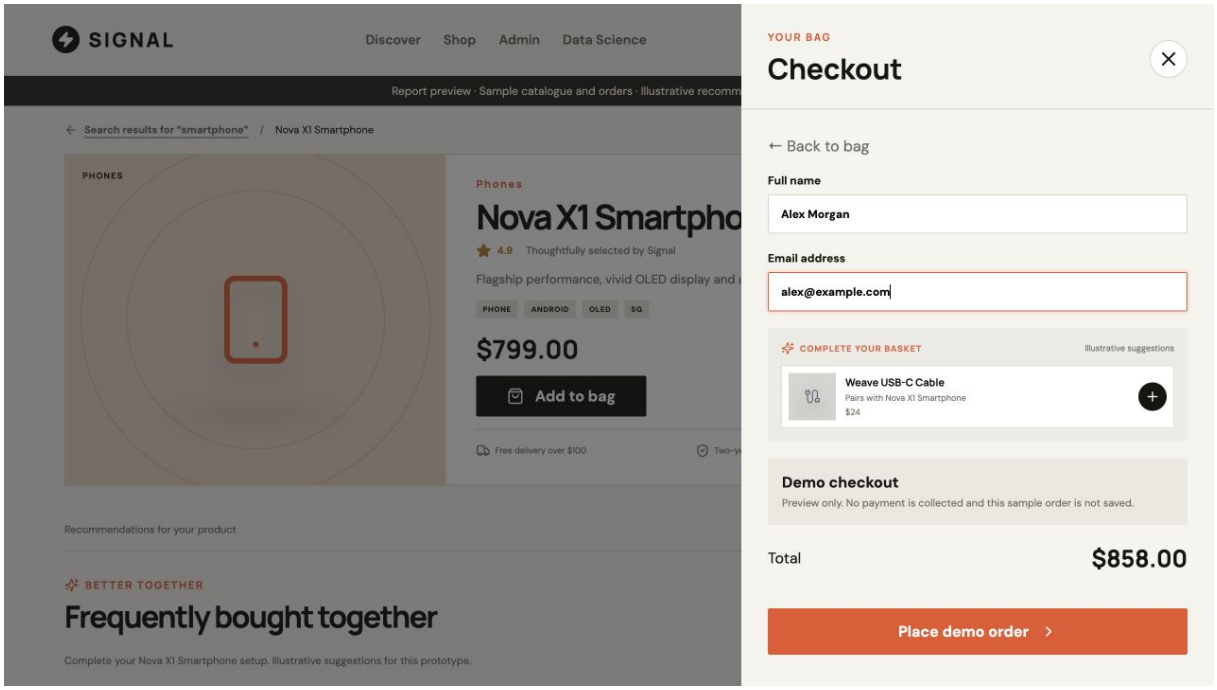


Figure 10: Demo checkout with minimal contact information and order total.

5.1.9 Order confirmation

Related requirements: FR-B02.

Figure 11 confirms the demonstration order with an order number and total. The bag is cleared; preview confirmation does not change stored orders or sales figures.

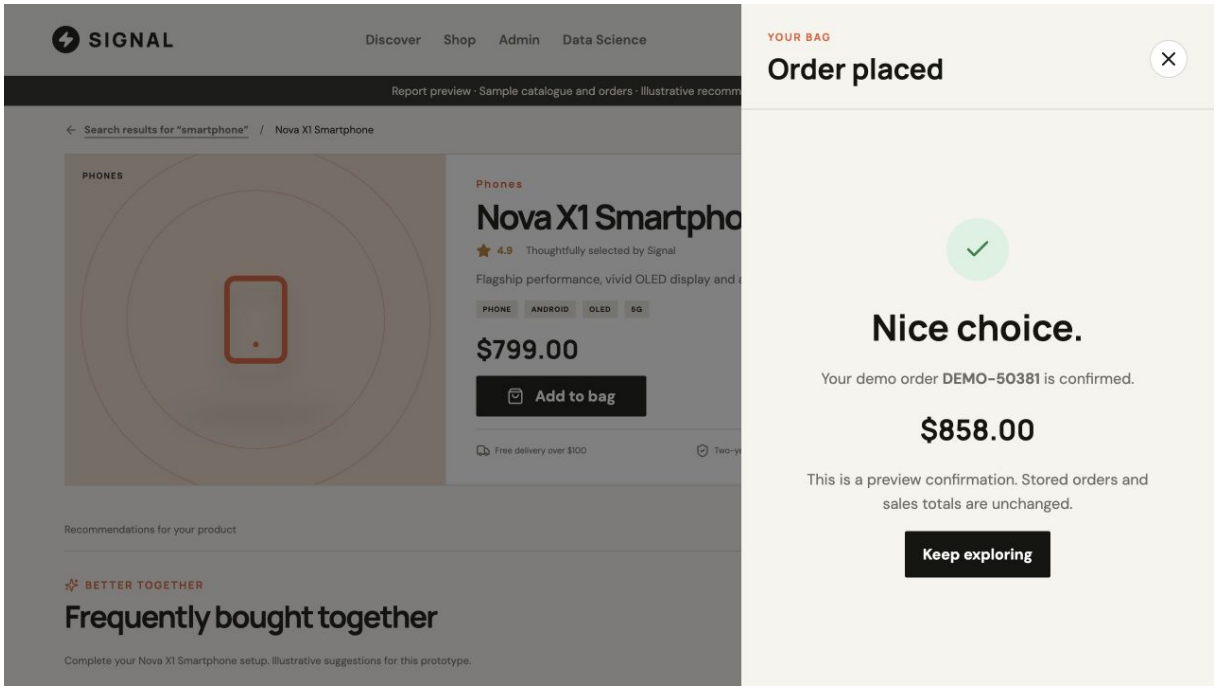


Figure 11: Simulated order confirmation after completing checkout.

5.2 Administration dashboard and order inspection

Table 22 maps the administration views to Table 3. Preview sales use consistent USD amounts and exclude canceled orders. Catalogue forms are shown for interface review; saving and withdrawal require live mode.

Table 22: Administration screens and related requirements.

Screen	Information and actions	Requirements
Catalogue and form	Product list, search, edit/withdraw actions and product data entry (Figures 12–13).	FR-A01
Sales overview	Revenue, orders, average order value, best sellers and trends with period selection (Figure 14).	FR-A03
Orders	Search, status filtering, pagination and order selection (Figure 15).	FR-A02
Order inspection	Customer, date, status, line items, purchase-time prices and totals (Figure 16).	FR-A02, FR-B03

5.2.1 Catalogue management

Related requirements: FR-A01.

Figure 12 shows the administrator's product list with search, category filtering and edit/withdraw actions. Withdrawing a product in live mode retains historical order items.

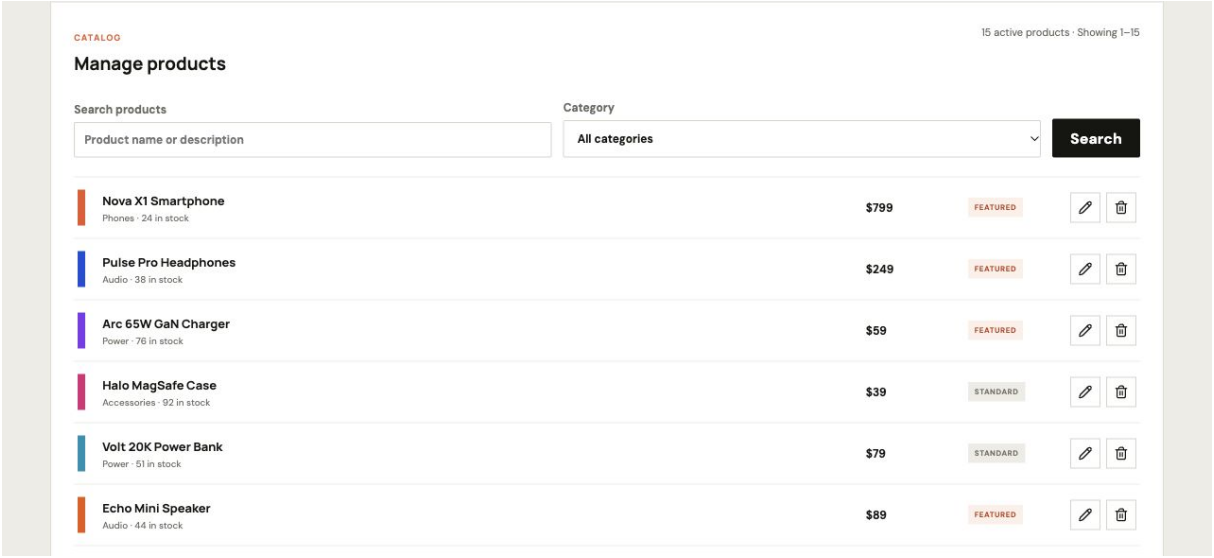


Figure 12: Administrator product list with catalogue-management actions.

5.2.2 Product information form

Related requirements: FR-A01.

Figure 13 shows sample data entered for a new product. The form covers name, description, price, stock, category and tags; editing uses the same fields.

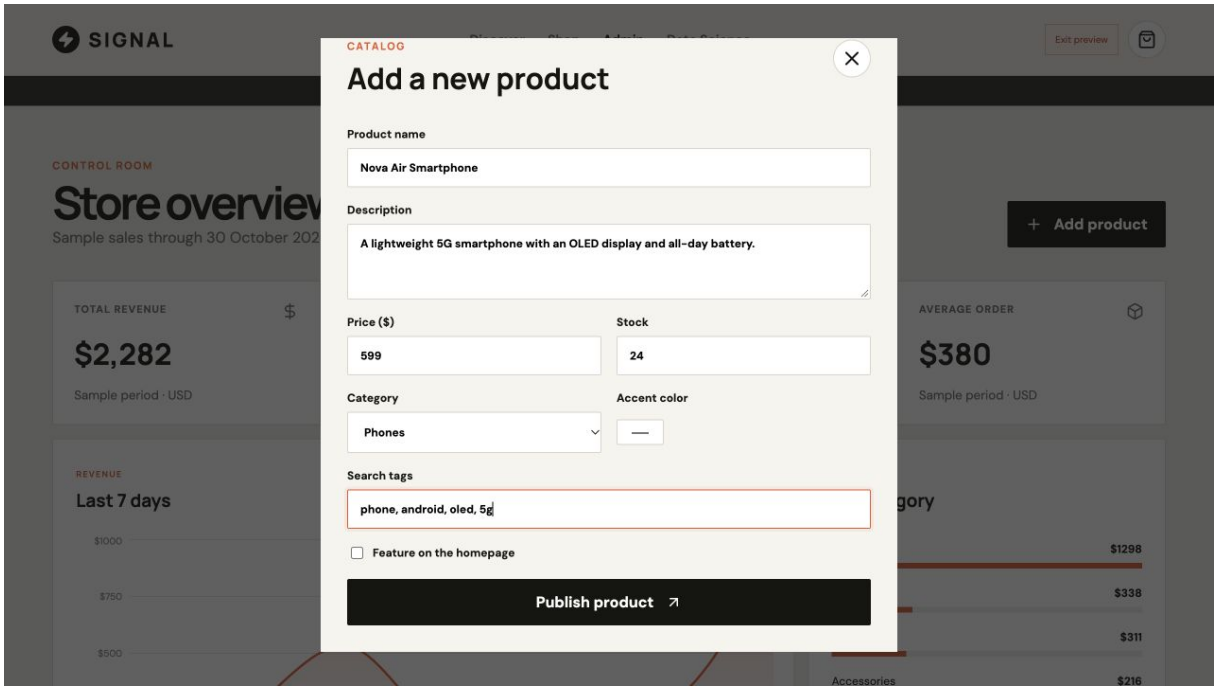


Figure 13: Product-entry form populated with demonstration values.

5.2.3 Sales overview

Related requirements: FR-A03.

Figure 14 displays sales indicators and trends for the selected seven-day or thirty-day period. These are sample-order aggregates; warehouse-derived reporting remains future work.

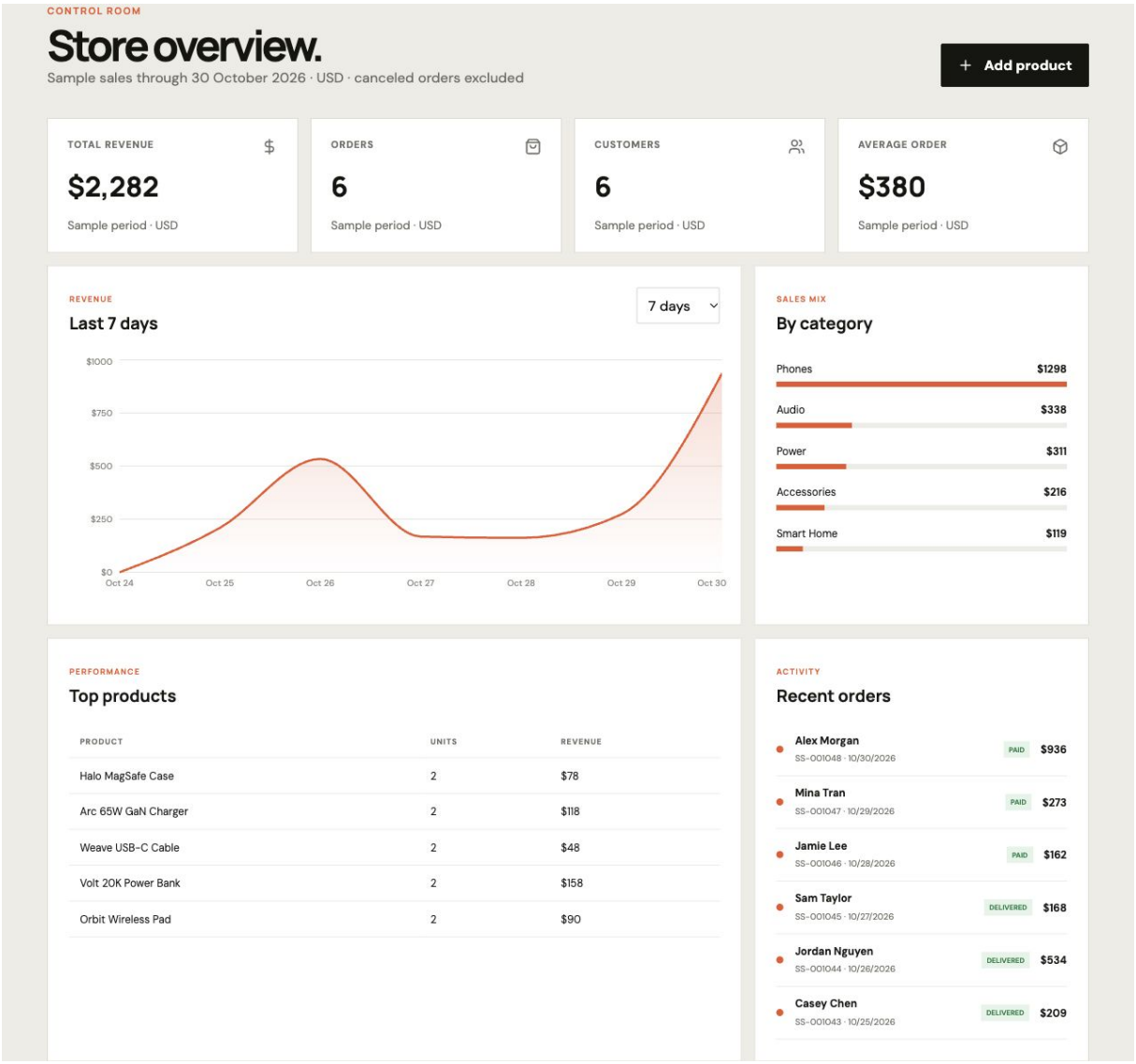


Figure 14: Sales overview with period selection and sample indicators.

5.2.4 Order browsing and selection

Related requirements: FR-A02.

Figure 15 lets the administrator search, filter and paginate orders. Selecting a record opens the inspection page in Section 5.2.5.

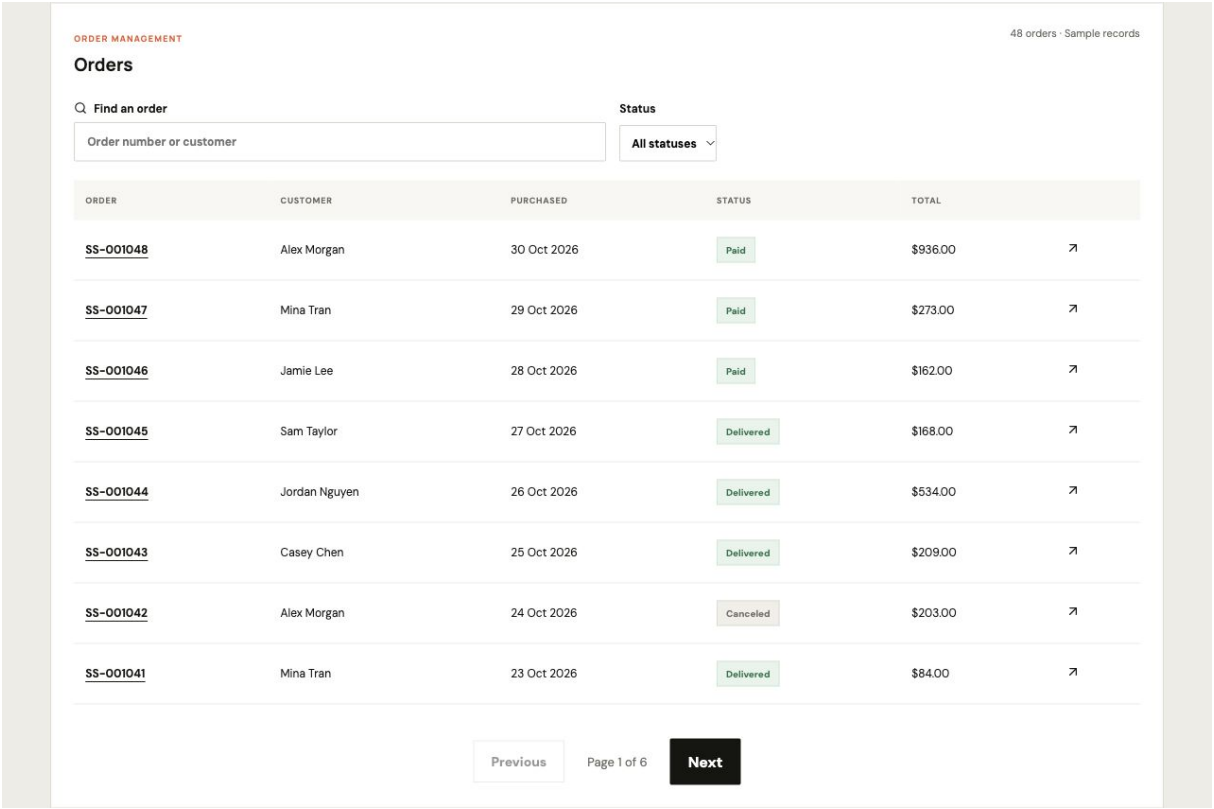


Figure 15: Searchable order records linked to inspection pages.

5.2.5 Order inspection

Related requirements: FR-A02, FR-B03.

Figure 16 shows a sample order's customer, status, purchase date and line items. In live mode, inspection reads stored operational records and retains purchase-time names, quantities and prices.

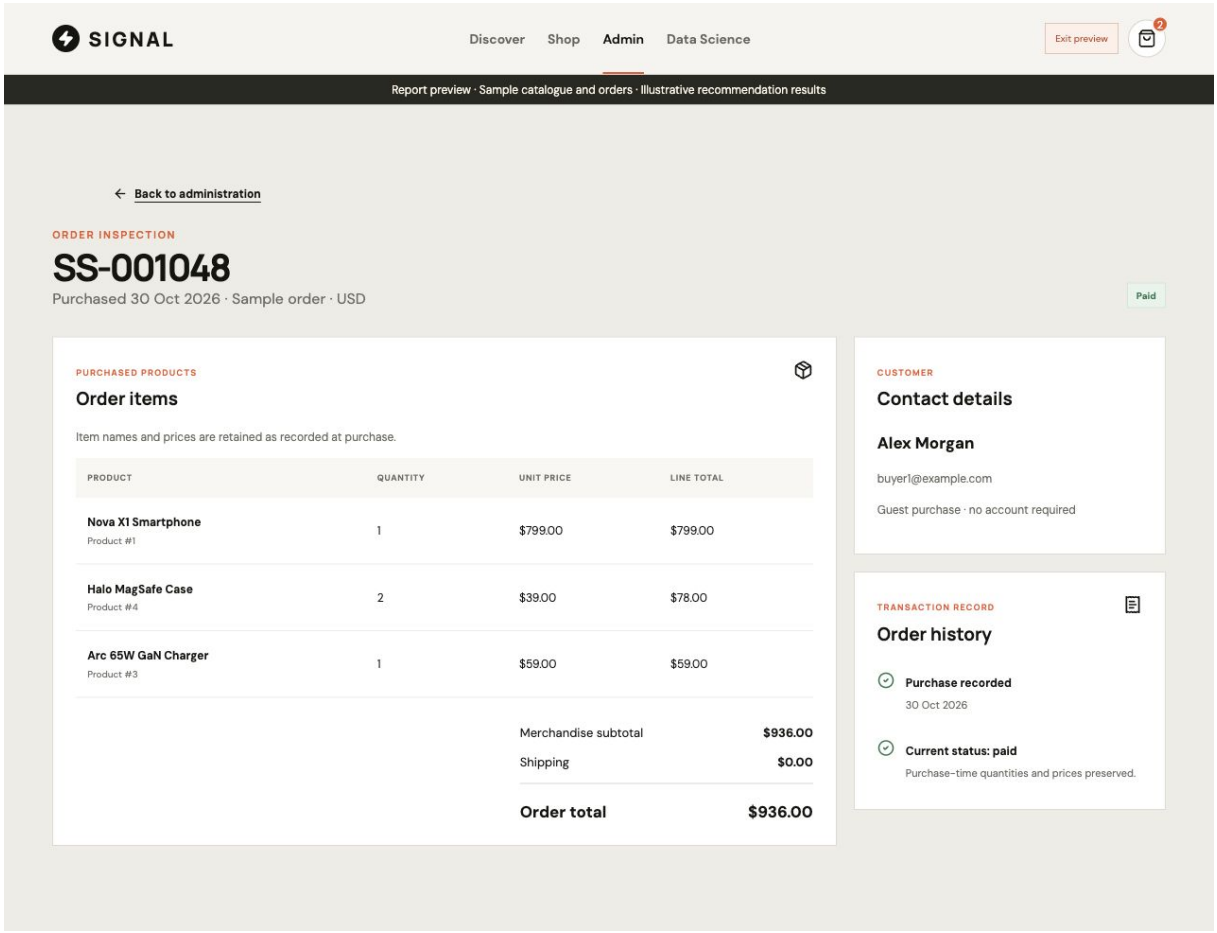


Figure 16: Sample order details with purchase-time quantities and prices.

5.3 Data-scientist workspace

The retained overview illustrates version traceability. External scripts and notebooks remain responsible for the eventual data preparation and training.

5.3.1 Dataset and model versions

Related requirements: FR-D02, FR-D06, FR-D10.

Figure 17 links sample model versions to dataset references, preparation times and statuses. It visualises experiment metadata without performing dataset extraction or retraining.

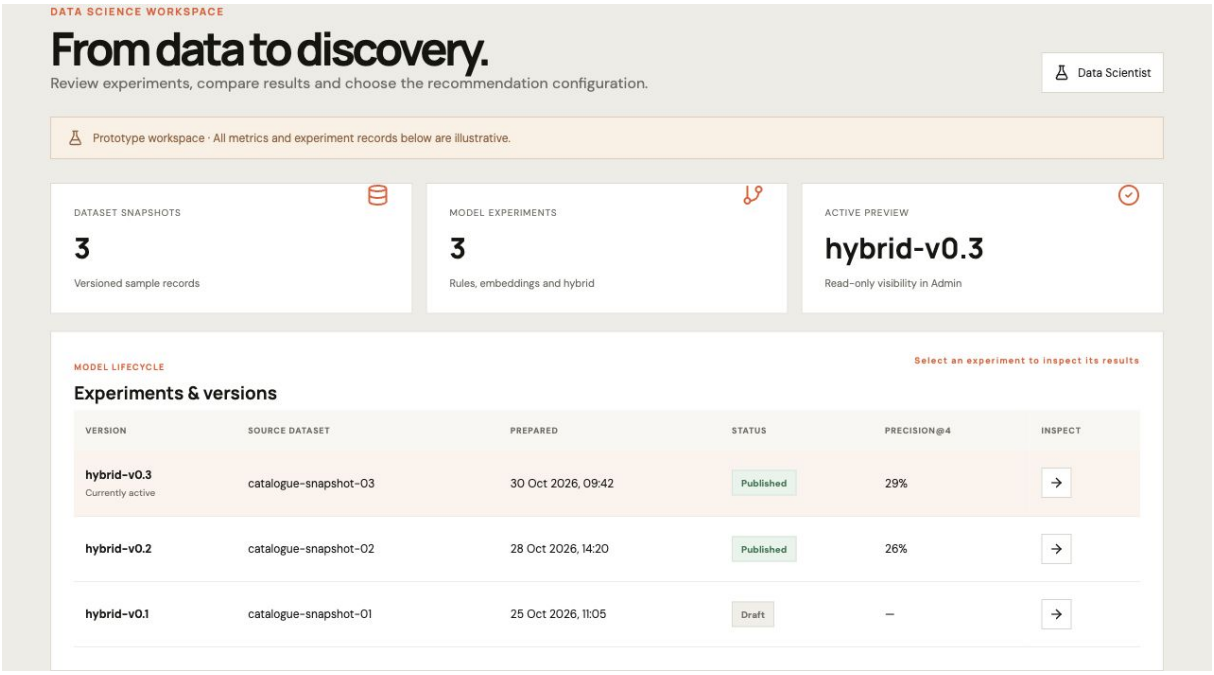


Figure 17: Sample dataset and model versions in the data-scientist workspace.

5.4 Prototype boundary and remaining implementation

The requirement links describe interface coverage, not completion of the analytical requirements. Dedicated active-model status (FR-A04) and detailed evaluation, publication and activation views (FR-D07–FR-D09) are omitted from the mockups.

Table 23: Prototype scope and remaining work.

Area	Demonstrated	Remaining implementation
Recommendations	Explained curated pairings, four alternatives and whole-basket context.	Learned rules, word embeddings and hybrid ranking (FR-B04, FR-B05, FR-D03–FR-D05).
Model overview	Sample dataset references, model versions and statuses.	Versioned datasets/artefacts, measured evaluation, publication, activation and role enforcement (FR-D02, FR-D06–FR-D10).
Purchases and catalogue	Buyer flow, order inspection and catalogue forms.	Preview checkout is not persisted; catalogue saving/withdrawal require live mode.
Reports and warehouse	Consistent sample sales; no warehouse-management mockup.	ETL, analytical structures and warehouse-derived reports (FR-D01, FR-W01–FR-W06).

References

[1] Olist, *Brazilian E-Commerce Public Dataset by Olist*. Kaggle. Available: `https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce`. Accessed: 10 October 2026.

[2] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules," in *Proc. 20th Int. Conf. Very Large Data Bases (VLDB)*, 1994, pp. 487–499.

[3] J. Han, J. Pei, and Y. Yin, "Mining frequent patterns without candidate generation," in *Proc. ACM SIGMOD*, 2000, pp. 1–12.

[4] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv:1301.3781*, 2013.

[5] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *Trans. Assoc. Comput. Linguistics*, vol. 5, pp. 135–146, 2017.

[6] F. Ricci, L. Rokach, and B. Shapira, Eds., *Recommender Systems Handbook*, 2nd ed. Springer, 2015.

[7] A. Cockburn, *Writing Effective Use Cases*. Addison-Wesley, 2000.

[8] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. Wiley, 2013.